

Chế độ Biểu đồ (Graph - TF 1.x)

- Yêu cầu định nghĩa đồ thị trước.
- Mở `tf.Session` để thực thi (Graph Execution).
- Quá trình debug vô cùng tàn bạo.



Chế độ Tức thời (Eager - TF 2.x)

- Phương thức hoạt động mặc định hiện tại.
- Thực thi tức thì: Mã chạy dòng nào có kết quả ngay.
- Gỡ lỗi dễ dàng bằng `print()` truyền thống.



Khuyết điểm của Eager Mode: Chậm hơn nhiều so với Graph do không thể tối ưu song song hóa phần cứng.

Gemini Notebook

`@tf.function”: Phép màu của AutoGraph

TensorFlow 2 mang đến giải pháp dung hòa bằng AutoGraph! Bạn viết code Python bình thường, chỉ cần đánh dấu `@tf.function`. TF sẽ tự động dịch thành đồ thị C++ ẩn bên dưới.

```python
def cube(x):
    return x ** 3

# Tinh toan - Nhanh cho Test nhưng cham khi huan luyện
cube(tf.constant(2.0))

# Khai bao TF Function bang decorator
@tf.function
def tf_cube(x):
    return x ** 3

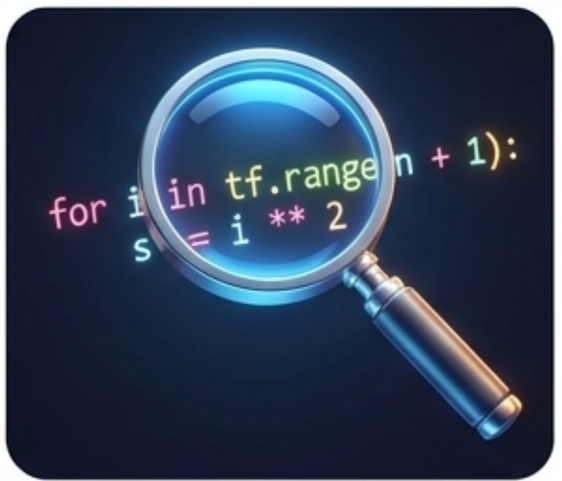
# tf_cube tro thanh mot computational graph!
tf_cube(tf.constant(2.0))
```



Cầu nối chuyển đổi tự động sang Graph C++

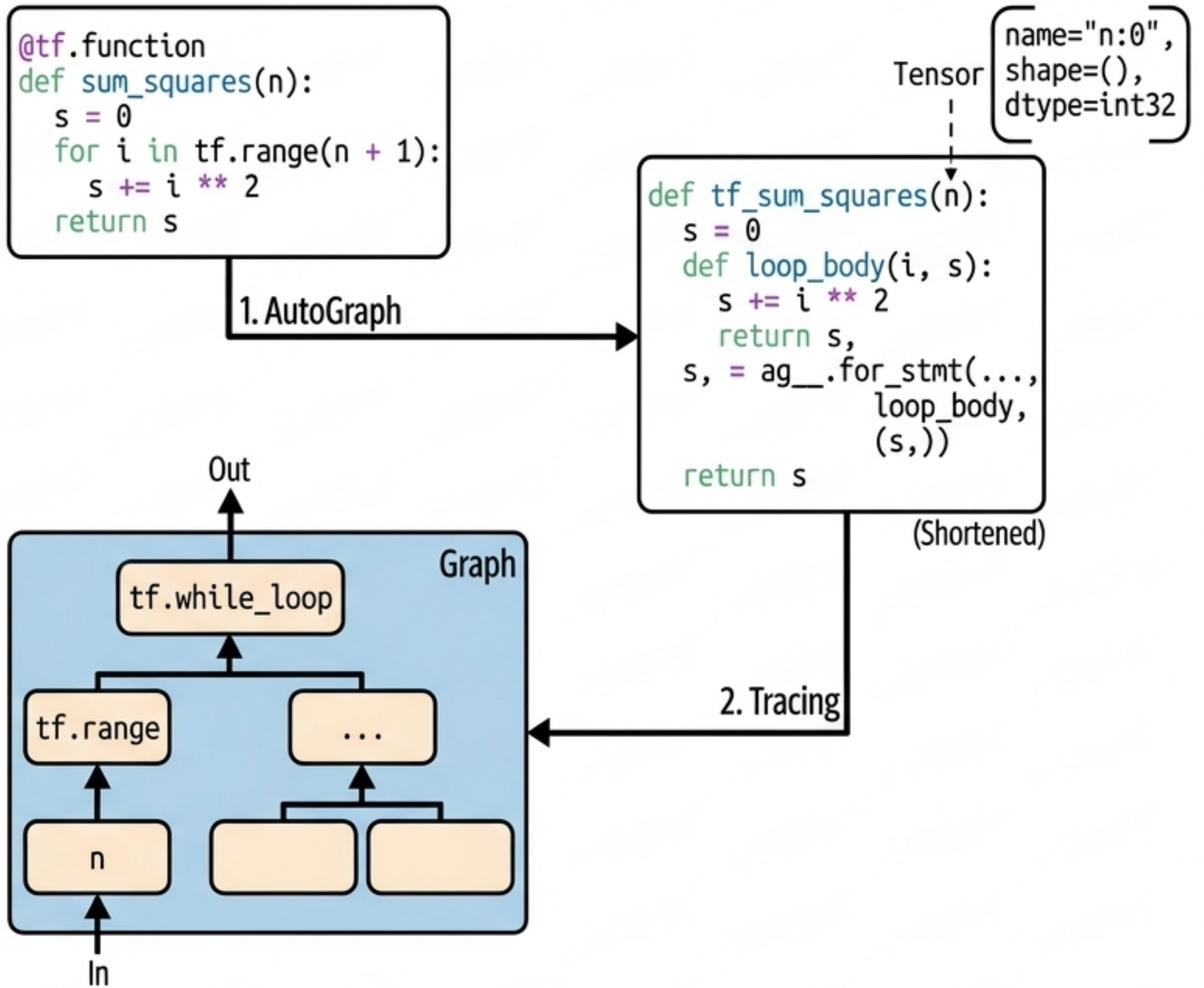
Gemini Notebook

Cách AutoGraph tạo Biểu đồ và Tracing (1/2)



Mã Python được quét, tạo Abstract Syntax Tree (Cây cú pháp trừu tượng).

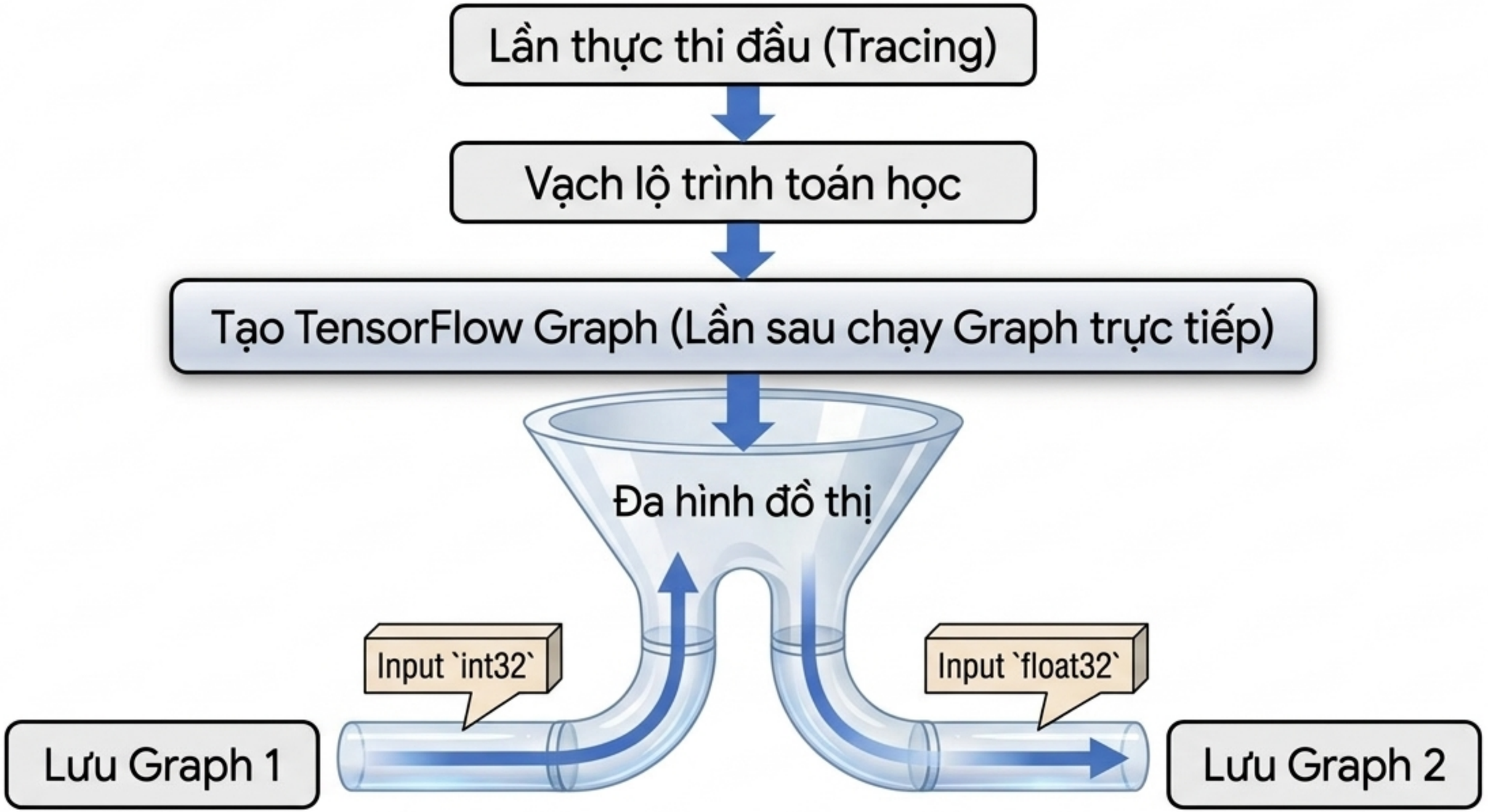
Sinh ra hàm Python nâng cấp gọi các toán tử điều khiển lõi của TF như `**tf.cond**, `**tf.while_loop**`.



*Hình 12-4: Cách TensorFlow tạo biểu đồ bằng AutoGraph và

Gemini Notebook

Cách AutoGraph tạo Biểu đồ và Tracing (2/2)



Gemini Notebook

Giới hạn và Quy tắc sống còn của TF Functions (1/2)



Side effects: Hàm `print()` hoặc thao tác IO chỉ chạy một lần duy nhất ở giai đoạn **Tracing**, không nằm trong Đồ thị.

```python
@tf.function
def my_func(x):
    print("Ham dang duoc trace:", x.dtype)
    return x + 1

my_func(tf.constant(1.0))  # In log, Tao Graph float32
my_func(tf.constant(2.0))  # KHONG in log, Tai sụ dung Graph float32
my_func(tf.constant(10))   # In log, Tao Graph int32
```

Biểu đồ đã được trace, hàm Python **KHÔNG được chạy lại!**



Muốn in log động? Khuyên dùng `tf.print()`.

Gemini Notebook

Code Lab

Giới hạn và Quy tắc sống còn của TF Functions (2/2)



Biến ngoài hàm / List Python:

`list.append()` bị "đóng băng" ở trace.

Giải pháp: Hãy dùng `tf.TensorArray`.

```python
@tf.function
def add_random_noise(x):
    # SAI LAM: Gia tri ngau nhien bi dong bang
    # noise = np.random.randn()

    # CHUAN XAC: Gia tri ngau nhien sinh ra moi khi goi Graph
    noise = tf.random.normal(shape=[])
    return x + noise
```



Giá trị ngẫu nhiên bị đóng băng



Giá trị sinh ra mới mỗi khi gọi Graph

Gemini Notebook

Code Lab

Vòng lặp huấn luyện tùy chỉnh

Sự Tiện lợi



Sự Quyền năng



VS

Keras `model.fit()'

Rất tiện lợi, nhưng đóng hộp. Không phải kiến trúc nào cũng xài được (VD: mạng GAN huấn luyện Generator 1 bước, Discriminator 1 bước).

Custom Training Loop

Mất công tự viết vòng lặp, nhưng phá vỡ mọi giới hạn. Cho phép tùy biến và can thiệp ở cấp độ sâu nhất.

Gemini Notebook

Code Lab
`tf.GradientTape(): Tính đạo hàm tự động

TensorFlow cung cấp GradientTape đóng vai trò 'băng ghi âm' lưu vết Tensor để lan truyền ngược (Backpropagation) tính đạo hàm tự động.



```python
def f(w1, w2):
    return 3 * w1 ** 2 + 2 * w1 * w2

w1, w2 = tf.Variable(5.), tf.Variable(3.)

with tf.GradientTape() as tape:
    z = f(w1, w2)

# Yeu cau Tape tinh dao ham Z theo bien w1, w2
gradients = tape.gradient(z, [w1, w2])
print(gradients)
# Ket qua: dz/dw1=36, dz/dw2=10
```

Đạo hàm chính xác: dz/dw1 và dz/dw2

Gemini Notebook

Code Lab

GradientTape nâng cao

- **Mặc định:** Chỉ theo dõi biến `tf.Variable` (Không theo dõi `tf.constant`).
- Khi gọi phương thức `gradient()`, bộ nhớ Tape tự động bị xóa!
- **Giải pháp:** Tính nhiều lần đạo hàm (VD: Hessian), dùng cờ `persistent=True`.

```
1  with tf.GradientTape(persistent=True) as tape:
2      z = f(w1, w2)
3
4  dz_dw1 = tape.gradient(z, w1)
5  dz_dw2 = tape.gradient(z, w2) # Văn OK
6
7  # Giai phong bò nhớ bằng tay (bat buóc)
8  del tape
```

Cẩm trịnh, nhớ chi lùng persistent=True

Hành động bắt buộc: Giải phóng bộ nhớ thủ công khi dùng **persistent=True**



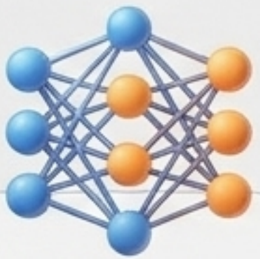
Gemini Notebook

Code Lab

Viết Vòng lặp huấn luyện - Khởi tạo

Bước 1: Chuẩn bị danh sách nguyên liệu. Tạo mô hình, cấu hình siêu tham số, Optimizer, Metric và Dữ liệu.

1. Khởi tạo Mô hình



```
l2_reg = keras.regularizers.l2(0.05)
model = keras.models.Sequential([
   keras.layers.Dense(30, activation="elu", kernel_regularizer=l2_reg),
   keras.layers.Dense(1, kernel_regularizer=l2_reg)
])
```

2. Chuẩn bị Dữ liệu



```
batch_size = 32
train_set = tf.data.Dataset.from_tensor_slices((X_train, y_train))
train_set = train_set.shuffle(len(X_train)).batch(batch_size)
```

3. Khởi tạo Optimizer



```
optimizer = keras.optimizers.Nadam(learning_rate=0.01)
```

4. Khởi tạo Loss & Metrics



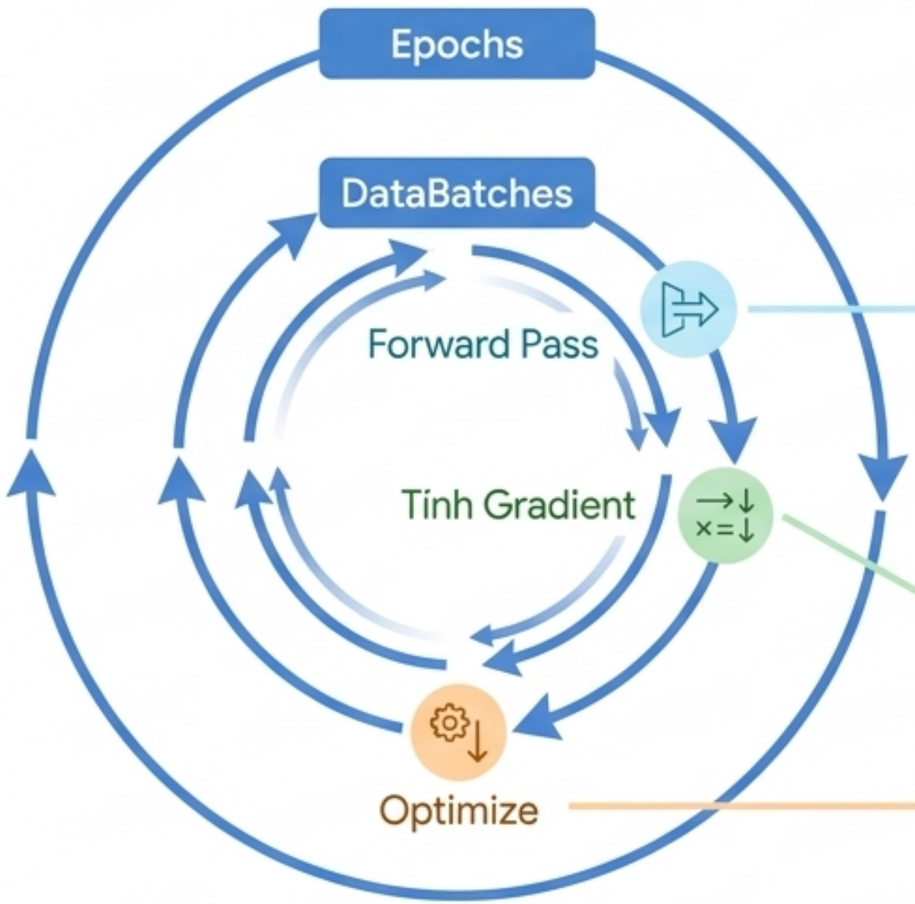
```
loss_fn = keras.losses.mean_squared_error
mean_loss = keras.metrics.Mean()
```

Gemini Notebook

Code Lab

Viết Vòng lặp huấn luyện - Thực thi (Epochs & Steps)

Lặp qua Epochs, trong Epochs lặp qua DataBatches.




```python
epochs = 5
for epoch in range(epochs):
  for step, (X_batch, y_batch) in enumerate(train_set):

    # 1. Forward pass ghi bang Tape
    with tf.GradientTape() as tape:
        y_pred = model(X_batch, training=True)
        main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
        loss = tf.add_n([main_loss] + model.losses)

    # 2. Tinh Gradient theo model variables
    gradients = tape.gradient(loss, model.trainable_variables)

    # 3. Kich hoat phuong thuc cap nhat trong so (Optimize)
    optimizer.apply_gradients(zip(gradients,
model.trainable_variables))
```

Gemini Notebook

Code Lab

Cấu trúc hoàn chỉnh và Những lưu ý quan trọng

Khi tự viết vòng lặp, BẠN phải chịu trách nhiệm đảm bảo mọi thứ chính xác. Đừng quên 3 điểm chết người này:



Truyền cờ `training=True`

Bắt buộc truyền vào lệnh dự đoán của model để các lớp như BatchNormalization và Dropout hoạt động chính xác trong quá trình huấn luyện.



Quản lý Metrics thủ công

Phải liên tục theo dõi bằng `update_state(loss)`, in ra màn hình vào cuối mỗi Epoch và quan trọng nhất: gọi `reset_states()` để tránh cộng dồn sai.



Xử lý biến hệ thống (Non-trainable variables)

Tự tay viết logic cập nhật các biến không thể huấn luyện, ví dụ như giá trị trung bình trượt (moving averages) của lớp Batch Norm.

Gemini Notebook

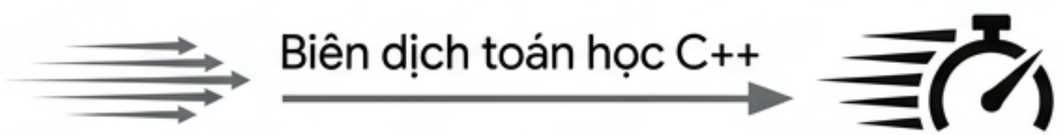
Tối ưu hóa Vòng lặp với **@tf.function**

Vòng lặp `for` Python qua các DataBatches cực kỳ chậm. **Lời khuyên: Đóng gói toàn bộ quá trình xử lý 1 Bước (1 Step)** thành **@tf.function** để tăng tốc biên dịch toán học C++.



```python
@tf.function
def train_step(X_batch, y_batch):
    with tf.GradientTape() as tape:
        y_pred = model(X_batch, training=True)
        main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
        loss = tf.add_n([main_loss] + model.losses)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

# Trong vong lap chi goi train_step(X_batch, y_batch)
```



Gemini Notebook

Tóm tắt: **4 Trụ cột TensorFlow Nâng cao**



Cốt lõi TensorFlow

Đại lượng Tensors cực kỳ giống NumPy, tối ưu hóa sức mạnh tính toán song song trên đa phần cứng (GPU/TPU).



Tùy biến Keras

Cho phép người học và nghiên cứu làm mới hoàn toàn và tự xây dựng các Losses, Metrics, Layers, Layers, Models chuyên biệt.



AutoGraph `@tf.function`

Giải pháp dung hòa hoàn hảo: Giải quyết sự linh hoạt của kịch bản Python với tốc độ biên dịch cực nhanh của đồ thị C++.



`tf.GradientTape`

Đưa quyền kiểm soát hoàn toàn tới luồng huấn luyện, mở khóa khả năng tính đạo hàm tự động cho các nghiên cứu mạng nơ-ron phức tạp nhất.

Gemini Notebook